

Lab 2: Decision Tree Classifier – Multi-Dataset Analysis

ID3 Decision Tree implemented from scratch (PyTorch and NumPy) and compared across three datasets

Name: Mohamed Riaz

Registration Number: PES1PGE25DS037

Department: Data Science & AI

Course: UX25CS635A – Supervised Machine Learning: Classification

Submission Date: 14 August 2026

1. Introduction

This lab implements the ID3 Decision Tree algorithm from scratch (no scikit-learn model calls) using both **PyTorch tensors** and **NumPy arrays**, and applies it to three structurally different categorical datasets: Mushroom Classification, Tic-Tac-Toe Endgame, and Nursery School recommendation. Only four building-block functions were implemented (*get_entropy_of_dataset*, *get_avg_info_of_attribute*, *get_information_gain*, *get_selected_attribute*); the provided test harness recursively calls them to grow the full tree, evaluate it, and report complexity metrics.

2. Dataset Descriptions

Dataset	Instances	Features	Class distribution	Balance
Mushroom Classification	8,124	22 (all categorical)	2 (edible 51.8% / poisonous 48.2%)	Balanced
Tic-Tac-Toe Endgame	958	9 (each cell: x/o/b)	2 (win 65.3% / not-win 34.7%)	Mild imbalance
Nursery School	12,960	8 (all categorical)	5 (33.3% / 32.9% / 31.2% / 2.5% / 0.02%)	Severe imbalance

All three datasets are purely categorical and are label-encoded column-by-column before tree construction, with the last column always treated as the target. Nursery is the most imbalanced by far: its *recommend* class has only 2 of 12,960 rows (0.02%), which turns out to matter a great deal for macro-averaged metrics (Section 4).

3. Methodology

ID3 core functions: *get_entropy_of_dataset* computes $-\sum(p_i \cdot \log_2 p_i)$ over the class distribution of the target column. *get_avg_info_of_attribute* partitions the data by every unique value of one attribute and returns the sample-weighted sum of each partition's entropy. *get_information_gain* subtracts that weighted entropy from the dataset entropy and rounds to 4 decimals. *get_selected_attribute* computes the gain for every remaining attribute and returns both the full gain dictionary and the index of the attribute with the maximum gain.

Tree construction (test harness): starting from the training split, the harness recursively splits on the attribute with the highest information gain, stopping when a node is pure, an attribute budget of *max_depth=7* is reached, all attributes are used, or no attribute yields positive gain – in each stopping case the majority class of that node is returned as a leaf. Both frameworks implement identical logic; the only difference is PyTorch tensor ops (*torch.unique*, *torch.log2*) vs. NumPy array ops (*np.unique*, *np.log2*).

Train/test split: an 80/20 shuffle-split with a fixed seed (42), but the two frameworks shuffle with their own RNGs (*torch.randperm* vs. *np.random.permutation*), so the two implementations do not see literally the same split – this is the source of the small differences reported in Section 4, not a bug in either implementation.

4. Results and Analysis

4.1 Performance Comparison

Dataset	Framework	Accuracy	Precision(w)	Recall(w)	F1(w)	Precision(macro)	F1(macro)
Mushroom	NumPy	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Mushroom	PyTorch	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000

Tic-Tac-Toe	NumPy	0.8836	0.8827	0.8836	0.8822	0.8784	0.8680
Tic-Tac-Toe	PyTorch	0.8730	0.8741	0.8730	0.8734	0.8590	0.8613
Nursery	NumPy	0.9887	0.9888	0.9887	0.9887	0.9577	0.9576
Nursery	PyTorch	0.9867	0.9876	0.9867	0.9872	0.7604	0.7628

Mushroom is solved perfectly by both implementations. Tic-Tac-Toe sits around 87-88% accuracy for both. Nursery reaches ~98-99% weighted accuracy in both, but its **macro-averaged metrics diverge sharply** (NumPy 0.958 vs. PyTorch 0.760 macro F1) – see Section 4.4 for why this is a class-imbalance/random-split effect rather than an algorithmic difference.

4.2 Tree Characteristics Analysis

Dataset	Framework	Max Depth	Total Nodes	Leaf Nodes	Root attribute (gain)
Mushroom	NumPy	4	29	24	odor (0.9048)
Mushroom	PyTorch	4	29	24	odor (0.9083)
Tic-Tac-Toe	NumPy	7	260	165	middle-middle-square (0.0909)
Tic-Tac-Toe	PyTorch	7	281	180	middle-middle-square (0.0834)
Nursery	NumPy	7	983	703	health (0.9597)
Nursery	PyTorch	7	952	680	health (0.9595)

Both frameworks independently select the **same root attribute** for every dataset (odor / middle-middle-square / health) with matching information gain to 2-3 decimal places, which is strong evidence the two implementations compute entropy/gain identically; the small differences in node counts (e.g. 260 vs. 281 for Tic-Tac-Toe) come from the different train/test shuffles producing slightly different subtrees below the root, not from a difference in the ID3 logic itself.

Mushroom needs only depth 4 and 29 nodes to reach 100% accuracy – *odor* alone almost perfectly separates the classes, and only one branch (odor = 'none') needs a second split on spore-print-color. Tic-Tac-Toe and Nursery both grow to the maximum allowed depth of 7 with far more nodes (260-983), reflecting that no small set of attributes is individually decisive and the tree must chain many splits together to isolate each class.

4.3 Dataset-Specific Insights

Mushroom Classification:

- Feature importance: *odor* dominates (gain ≈ 0.90 , more than 5 $\times$ the next-best attribute), followed by spore-print-color for the residual 'no odor' branch.
- Class distribution: balanced (51.8% / 48.2%), so accuracy is a trustworthy metric here.
- Decision pattern: almost a single-level lookup on odor, with only one deeper branch – a biologically plausible pattern (smell is a well-known practical heuristic for mushroom toxicity).
- Overfitting indicators: none observed – depth 4 with 100% test accuracy on a held-out 20% split indicates the classes truly are (near-)perfectly separable on these features, not memorization.

Tic-Tac-Toe Endgame:

- Feature importance: *middle-middle-square* is selected as root in both frameworks (the center square is well known to be strategically the most valuable position in Tic-Tac-Toe), but its gain (≈ 0.08 - 0.09) is an order of magnitude lower than mushroom's root gain – no single square is individually decisive.

- Class distribution: mildly imbalanced (65.3% win / 34.7% not-win).
- Decision pattern: deep, wide trees (260-281 nodes) that combine many board positions – consistent with the fact that a win/loss outcome genuinely depends on the joint configuration of multiple cells, not any one cell alone.
- Overfitting indicators: this is the dataset with the largest node count relative to its size (958 rows -> 260+ nodes, i.e. under 4 samples/node on average) alongside the lowest test accuracy ($\approx 87-88\%$) of the three – a classic small-data/high-node-count overfitting signature.

Nursery School:

- Feature importance: *health* is by far the strongest attribute (gain ≈ 0.96 , close to mushroom's odor), which matches domain intuition – health status was designed into the original ranking rules used to generate this dataset.
- Class distribution: severely imbalanced – 3 large classes (31-33% each) plus *very_recom* (2.5%) and *recommend* (0.02%, only 2 rows total).
- Decision pattern: very large tree (952-983 nodes) needed to carve out the rare classes from the three dominant ones.
- Overfitting indicators: weighted accuracy stays high ($\approx 98-99\%$) purely because the three big classes are easy, while macro metrics (which weight all 5 classes equally) expose the real difficulty – the extremely rare *recommend* class (2 rows) is essentially unlearnable and highly sensitive to which split a random shuffle happens to put it in.

4.4 Manual (PyTorch) vs. Manual (NumPy) Comparison

Since scikit-learn model calls are disallowed, both implementations are 'manual' ID3 built on the same four formulas; the comparison here is PyTorch-tensor vs. NumPy-array arithmetic rather than 'from-scratch vs. library'. On Mushroom both are identical (100% / depth 4 / 29 nodes). On Tic-Tac-Toe both pick the same root and reach similar accuracy (87-88%) with slightly different tree sizes. On Nursery, weighted metrics agree closely (98.67-98.87% accuracy) but macro metrics diverge sharply (macro F1 0.9576 NumPy vs. 0.7628 PyTorch).

Root cause of the Nursery divergence: `torch.randperm(42)` and `np.random.permutation(42)` are different RNGs seeded the same way, so they produce different 80/20 splits of the 12,960 rows. The *recommend* class has only 2 rows in the entire dataset; depending on the split, those 2 rows can land such that the class is poorly represented in training, or is present in the test set but the tree never learned a leaf for it. Because macro-averaging treats all 5 classes equally, missing or misclassifying that one vanishingly-rare class swings the macro F1 by tens of percentage points, while the weighted metrics barely move because that class carries almost no weight (0.02% of samples). This is a textbook illustration of why macro metrics, not just accuracy, must be reported for imbalanced multi-class problems.

5. Comparative Analysis Report

a) Algorithm Performance

- **Highest accuracy:** Mushroom (100%), because *odor* alone is an almost perfect discriminator – a single categorical feature happens to be near-deterministic for the label, so even a shallow tree separates the classes exactly.
- **Effect of dataset size:** the largest dataset (Nursery, 12,960 rows) and the smallest (Tic-Tac-Toe, 958 rows) both needed the maximum depth of 7, but Tic-Tac-Toe's much smaller training set (766 rows) spread across 9 attributes with 3 values each produced a disproportionately large, sparse tree (under 4 samples per node on average) and the weakest accuracy of the three – small sample size relative to the attribute/value space directly hurts ID3's ability to estimate reliable splits.
- **Role of feature count:** Mushroom has the most features (22) but the fewest are actually needed (root alone nearly solves it), while Tic-Tac-Toe has the fewest features (9) but needs almost all of them combined to reach only ~87-88% – more features help only when at least one of them is individually informative; when information is spread thin across many weakly-informative features (as in Tic-Tac-Toe), extra features mostly add tree complexity, not accuracy.

b) Data Characteristics Impact

- **Class imbalance:** as shown in Section 4.4, imbalance does not hurt weighted accuracy/F1 (which stayed $\geq 98.6\%$ even on the 5-class, highly-skewed Nursery data) but it makes the tree's treatment of rare classes highly sensitive to the random train/test split, and it is macro-averaged metrics that reveal this fragility; a single 0.02%-frequency class can swing macro F1 by 0.1-0.2 depending purely on which split it lands in.

- **Binary vs. multi-valued features:** Tic-Tac-Toe's ternary (x/o/b) features gave the lowest per-attribute information gain (root gain $\approx 0.08-0.09$) and the weakest accuracy, while Nursery and Mushroom's attributes with more distinct categories (e.g. odor has 9 values, health has 3) produced far higher single-attribute gains and shallower, more accurate splits close to the root – attributes with more distinguishing categories tend to separate classes more cleanly per split, provided those categories are actually correlated with the label (as odor and health are, but individual board cells are not).

c) Practical Applications

- **Mushroom → food-safety / foraging-assistant tools:** a shallow, high-accuracy tree is exactly what's needed for a safety-critical, explainable rule ('if odor is X, it's poisonous') that a non-expert forager or a mobile app could apply instantly.
- **Tic-Tac-Toe → game-outcome / strategy analysis:** useful for teaching search/game theory concepts and for building simple heuristic game-evaluation functions, but the deep, low-gain-per-split tree shows ID3 is a poor fit for problems where the outcome depends on joint combinations of positions rather than individually informative features – a minimax/game-tree search is the more natural tool for this domain in practice.
- **Nursery → policy/admissions decision support:** the tree gives interpretable, auditable admission-priority rules ('if health is not_recom, recommendation = not_recom') which matters for a domain where decisions must be explainable to families and administrators; however, the rare classes need special handling (e.g. oversampling, or manual review flags) before deploying this in a real admissions pipeline, since ID3 alone will not reliably serve the 0.02%-frequency class.
- **Interpretability:** all three trees are fully interpretable path-by-path, which is ID3's core practical advantage over black-box models in domains (food safety, admissions) where a human must be able to justify the specific decision path for a given case.
- **Suggested improvements per dataset:** Mushroom – already near-optimal, could prune further for an even simpler rule set; Tic-Tac-Toe – needs feature engineering (e.g. encode rows/columns/diagonals as derived features) or a different algorithm (minimax) rather than more tree depth; Nursery – needs class-imbalance handling (oversampling the rare classes, or reporting per-class recall alongside accuracy) before the rare-class results can be trusted.

6. Conclusion

- The PyTorch and NumPy implementations of the four ID3 building blocks are numerically consistent: they select the same root attribute with matching information gain on all three datasets, confirming the entropy/gain formulas were implemented correctly in both frameworks.
- Dataset characteristics, not the framework, drive the observed performance differences: Mushroom's single highly-informative feature yields a near-trivial, perfect tree; Tic-Tac-Toe's weakly-informative, combinatorial features force a large tree with the weakest accuracy; Nursery's severe class imbalance makes weighted and macro metrics tell very different stories.
- Class imbalance is the single biggest risk factor observed in this lab: it does not lower aggregate accuracy, but it makes a model's behaviour on rare classes essentially unpredictable across random splits, which is a serious concern in decision-support domains like Nursery admissions where those rare cases may be exactly the ones needing the most careful review.